Mixture-of-Collaboration: Expert Collaboration via Learned Mediators in Sparse Transformer Architectures

Technical Report

Francisco Antonio
Auren Research, Independent

March 2026

Abstract

Mixture-of-Experts (MoE) Transformers increase total parameter capacity while keeping per-token compute sparse, but standard top-k routing still treats selected experts as mostly independent branches whose outputs are only merged at the end. This report presents *Mixture-of-Collaboration* (MoC), a sparse decoder-only Transformer that augments MoE with explicit expert interaction through a learned mediator, weight-shared iterative refinement inside each expert, and an optional adaptive-compute policy for reasoning depth and collaboration rounds.

We report three tiers of evidence. First, in a controlled 64M-parameter ablation on a single NVIDIA A10, standard sparse routing already improves validation perplexity from 67.68 (dense baseline) to 59.33, and fixed-depth MoC further improves it to 55.67 under matched training budget and nearly matched active parameters per token. Second, an adaptive-compute MoC variant reaches similar modeling quality (56.02 perplexity) while learning non-uniform internal compute allocation, averaging 2.54/3 reasoning steps and 0.47/2 collaboration rounds with roughly 40% estimated internal compute reduction. Third, the current implementation of MoC scales to a larger 197M-parameter sparse configuration without routing collapse, reaching validation perplexity 45.73 in an 8-layer run.

These results support a narrow but meaningful claim: mediator-based collaboration is promising on top of sparse routing, and the architecture remains stable when scaled beyond the initial small-model regime. At the same time, the evidence is still preliminary: the controlled comparison is single-seed, the larger run is not paired against an equally sized MoE baseline, and adaptive efficiency has not yet been validated with controlled end-to-end inference benchmarks. Code, configs, and run logs are public.

1 Introduction

Sparse Transformers based on MoE routing can substantially increase total parameter capacity without making every token pay full dense-compute cost [1–4]. In standard top-k MoE, a router selects experts and the selected experts process token states independently before a weighted merge. This design is efficient, but it also constrains information exchange among experts to a single late aggregation point.

An immediate idea is to let experts communicate directly. However, many communication-heavy variants introduce pairwise interactions whose cost scales with the number of experts, making them difficult to scale at high expert counts. MoC instead adopts a learned mediator hub: experts communicate through a shared latent state, so per-token collaboration remains linear in selected experts ($\mathcal{O}(K)$), not quadratic in total experts.

A second issue is compute allocation. Fixed-depth processing assumes all tokens require equal effort, but tokens in language modeling vary widely in difficulty. A uniform budget may over-compute trivial tokens and under-compute difficult ones. MoC therefore incorporates adaptive gates for both intra-expert reasoning depth and inter-expert collaboration rounds.

Contributions.

- We introduce **MoC**, a sparse MoE architecture with mediator-based expert collaboration at linear cost in selected experts.
- We integrate **iterative expert refinement** via weight-shared multi-step FFN updates and describe an **adaptive-compute** extension for reasoning depth and collaboration rounds.
- We provide a **controlled 64M ablation** separating dense, standard MoE, fixed-depth MoC, and adaptive MoC variants.
- We report an **updated same-scale implementation run** and a **larger 197M scaling probe**, together with routing diagnostics and reproducibility details.

This document is a technical report (not a peer-reviewed paper) intended to record architecture details and preliminary validation at small scale with reproducible settings.

2 Architecture

2.1 Overview

The implementation in `model_moc.py` is a decoder-only Transformer with Rotary Position Embeddings (RoPE), grouped-query attention (GQA), RMSNorm, and SwiGLU-style FFNs. Each transformer block applies:

$$\textit{Attention} \rightarrow \textit{Router} \rightarrow \textit{Expert Dispatch} \rightarrow \textit{MoC Collaboration} \rightarrow \textit{Fuse/Project} \rightarrow \textit{Residual}.$$

When `n_experts` is `None`, blocks use a dense FFN path. Otherwise, each block uses `MoCTopKExperts` for routing, capacity filtering, per-expert IRL computation, and optional collaboration/fusion.

2.2 Router and Expert Dispatch

Given flattened token states $x \in \mathbb{R}^{N \times d}$, routing logits are computed in FP32:

$$\ell = xW_{\text{gate}}^{\top}, \qquad p = \text{softmax}(\ell). \tag{1}$$

During training, Gaussian router noise (std 0.02 in experiments) perturbs logits for exploration. Top-k experts are selected per token and top-k probabilities are normalized for weighted aggregation.

Capacity is enforced via a vectorized priority-based keep/drop policy:

- flatten all token-expert slots ($N \times K$),
- stable sort by descending routing priority,
- regroup by expert,

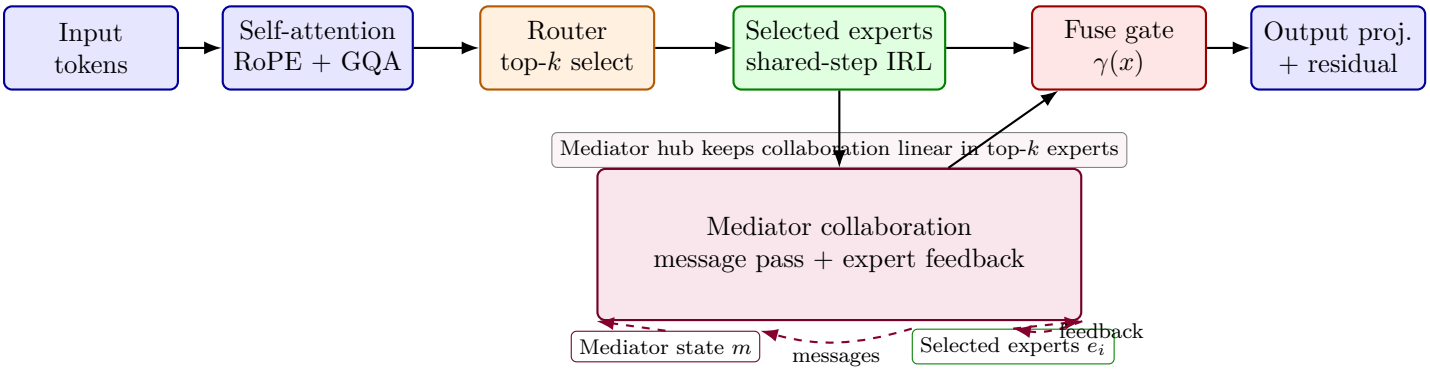

Figure 1: MoC block flow with mediator-centric collaboration. Sparse top-k routing chooses experts as in standard MoE, but the selected experts then exchange information through a learned mediator before gated fusion. The colored groups emphasize the progression from token processing (blue) to routing (orange), expert computation (green), collaboration (purple), and final fusion/output (red).

- keep up to per-expert capacity, drop the rest.

The code sets capacity from

$$C = 2^{\lceil \log_2(\lceil \text{capacity_factor} \cdot (NK/E) \rceil) \rceil}, \tag{2}$$

clipped to at most NK. Experiments use `capacity_factor=1.5`.

Auxiliary router regularization is:

$$\mathcal{L}_{\text{aux}} = \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \lambda_z \mathcal{L}_z + \lambda_{\text{drop}} \mathcal{L}_{\text{drop}}, \tag{3}$$

where $\lambda_{\text{bal}} = 0.01$, $\lambda_z = 0.001$, and $\lambda_{\text{drop}} = 0.001$ in MoE/MoC runs. The implementation computes:

$$\mathcal{L}_{\text{bal}} = E \sum_{e=1}^{E} \bar{p}_e \, \bar{f}_e, \tag{4}$$

$$\mathcal{L}_z = \mathbb{E}[\log \sum_e \exp(\ell_e)]^2, \tag{5}$$

$$\mathcal{L}_{\text{drop}} = \text{drop fraction}. \tag{6}$$

2.3 Iterative Reasoning Layers (IRL)

Each expert is a weight-shared iterative FFN:

$$h^{(0)} = x, \qquad h^{(t+1)} = h^{(t)} + \alpha \, \text{FFN}\Big(\text{RMSNorm}(h^{(t)} + x)\Big), \tag{7}$$

for $t = 0, \ldots, N_{\max} - 1$, with $\alpha = 1/\sqrt{N_{\max}}$ for stability. The FFN uses SwiGLU structure and reuses the same parameters at every step.

This recurrent refinement is related to equilibrium-style views of deep computation [8]: although MoC runs a finite number of iterations, the shared-weight formulation encourages fixed-point-like behavior while keeping parameter growth controlled.

2.4 MoC-Lite Collaboration

After expert computation, MoC-Lite refines token representation through a learned mediator state. For each token, let selected expert states be $\{e_i\}_{i=1}^{K}$ and weighted MoE merge be $\tilde{e} = \sum_i w_i e_i$.

Mediator initialization is:

$$m_0 = m_{\text{learned}} + W_{\text{ctx}} x \tag{8}$$

(or a fallback from $\tilde{e}$ if learned mediator is disabled).

One collaboration round computes:

$$q = W_q \,\text{Norm}(m), \tag{9}$$

$$k_i = W_k \,\text{Norm}(e_i), \quad v_i = W_v \,\text{Norm}(e_i), \tag{10}$$

$$a_i = \text{softmax}_i\left(\frac{q^\top k_i}{\sqrt{d}}\right), \tag{11}$$

$$u = \sum_i a_i v_i, \tag{12}$$

$$m' = m + \text{MLP}(\text{Norm}(m) + u). \tag{13}$$

If expert feedback is enabled (enabled in experiments), mediator-to-expert feedback is gated per expert:

$$g_i = \sigma(w_e^\top e_i + w_m^\top m'), \qquad e_i' = e_i + g_i \, W_{m\to e} m'. \tag{14}$$

Final fusion between mediator and weighted expert sum uses an input-conditioned gate:

$$\gamma = \sigma(W_f x + b), \qquad y = \gamma m + (1-\gamma) \sum_i w_i e_i'. \tag{15}$$

This hub topology keeps collaboration complexity linear in selected experts ($\mathcal{O}(K)$ per token) instead of all-expert pairwise interaction.

2.5 Adaptive Compute Gating

MoC includes two independent adaptive gates:

- **Reasoning gate** for IRL depth (0 to N_{max} steps).
- **Collaboration gate** for mediator rounds (0 to C_{max} rounds).

Each gate maps token state to logits over $\{0, 1, \ldots, S\}$ and applies temperature-scaled softmax. In `soft` mode (used in experiments), activity weights are cumulative tail probabilities, enabling compile-friendly dense execution:

$$\text{activity}_s = \sum_{j=s}^{S} P(\text{steps} = j). \tag{16}$$

In `hard` mode, the model selects an argmax step count and executes sparse per-token control flow.

Expected-step penalties encourage efficiency:

$$\mathcal{L}_{\text{step}} = \lambda_{\text{step}} \frac{\mathbb{E}[\text{steps}]}{S}, \tag{17}$$

with weight 0.001 for both reasoning and collaboration in MoC-vNext. Gate temperature is 1.0, gate noise std is 0.02.

Table 1: Controlled 64M experiment configurations. Effective batch is matched across runs.

Model	Expert routing	IRL setup	Collaboration	Adaptive compute	Total params	Active/token	Batch config
`a-vanilla`	none (dense FFN)	$N = 1$	off	off	35,789,952	35,789,952	bs=32, grad_acc=1
`b-moe`	$E = 4$, top-2, cap=1.5	$N = 1$	off (independent experts + weighted merge)	off	64,138,398	49,978,014	bs=32, grad_acc=1
`c-moc-fixed`	$E = 4$, top-2, cap=1.5	$N = 2$ fixed	mediator on, feedback on, 2 fixed rounds	off	64,131,468	49,971,084	bs=16, grad_acc=2
`d-moc-adaptive`	$E = 4$, top-2, cap=1.5	$N \leq 3$, adaptive soft gate	mediator on, feedback on, up to 2 rounds	reasoning + collaboration adaptive on	64,175,358	49,996,494	bs=16, grad_acc=2

3 Experimental Setup

We now distinguish between three experiment groups:

1. a controlled 64M-parameter ablation on an NVIDIA A10 (dense, MoE, fixed-depth MoC, adaptive MoC),
2. an updated same-scale MoC run using the current implementation,
3. a larger scaling probe at approximately 197M trainable parameters.

The first group is the cleanest architectural comparison. The latter two are useful for implementation validation and scaling behavior, but they should not be interpreted as architecture-only ablations because the code path also changed.

3.1 Controlled 64M Ablation

All four controlled runs share the same base decoder backbone:

- `d_model=384`, `n_layers=6`, `n_heads=6`, `n_kv_heads=6`
- `vocab_size=65536`, `max_seq_len=512`
- `ffn_hidden_multiplier=4.0`, `multiple_of=64`
- `dropout=0.0`, `rope_theta=10000.0`

Table 1 lists the controlled variants used in the initial A10 comparison.

Shared optimization settings for the controlled ablation:

- **Max steps:** 20,000
- **Sequence length:** 512
- **Effective batch:** 32 sequences per optimizer step
- **Approximate training budget:** 327.7M tokens
- **Optimizer:** AdamW, lr 3×10^{-4}, betas $(0.9, 0.95)$, weight decay 0.1
- **Schedule:** 500-step linear warmup then cosine decay to 1% of base lr
- **Router regularization:** `aux_loss_weight=0.01`
- **Router z-loss weight:** `0.001`
- **Drop penalty weight:** `0.001`

- **Router noise std:** `0.02`
- **Precision:** bf16 mixed precision
- **Seed:** 1337

The adaptive variant additionally uses soft adaptive-compute gates with temperature 1.0, gate noise std 0.02, and step-penalty weights 0.001 for both reasoning and collaboration.

3.2 Current Implementation Refresh and Scaling Probe

The current main-branch implementation differs from the earlier runs in both model internals and training plumbing. The most relevant changes include: persistent parameter-group learning-rate scaling for router parameters (the router group is held at **0.5× the base learning rate** throughout training rather than only at optimizer initialization), Q/K normalization inside attention, normalization inside the mediator MLP, a cleaner shard dataset path, and lower-overhead batch/prefetch and routing-log handling. Because these changes affect both optimization and systems behavior, same-scale improvements from the refreshed code should be treated as *implementation-level* evidence rather than a pure architectural ablation.

We report two additional runs from the updated code path:

- **Same-scale refreshed MoC:** approximately 64M parameters, intended as a sanity check against the earlier fixed-depth MoC run.
- **Larger sparse scaling probe:** approximately 197M parameters and 8 layers, intended to test whether routing and collaboration remain healthy beyond the initial small-model regime.

These later runs were executed on an NVIDIA L40S. Their throughput should therefore not be compared directly against the earlier A10 experiments.

3.3 Evaluation Protocol

Across runs we track:

- validation CE loss (reported as `val/val_ce_loss`)
- validation perplexity exp(validation CE)
- auxiliary-inclusive validation loss (reported separately as `best_val_loss` or `val/val_loss`)
- routing diagnostics: expert utilization, dead experts, drop rate, router entropy, and layer agreement
- adaptive metrics when enabled: average reasoning steps, average collaboration rounds, and estimated compute-saved percentage

Validation for the controlled runs was performed every 2,000 steps on 20 batches. The later runs follow the same logging style but are included here primarily as implementation and scaling evidence, not as part of the clean four-way ablation.

Table 2: Controlled 64M ablation using validation metrics. Main claims in the paper should be anchored to this table.

Model	Total Params	Active/Token	Val CE	Val PPL	Best Val Loss
`a-vanilla`	35,789,952	35,789,952	4.2147	67.68	4.2147
`b-moe`	64,138,398	49,978,014	4.0831	59.33	4.0933
`c-moc-fixed`	64,131,468	49,971,084	4.0195	55.67	4.0298
`d-moc-adaptive`	64,175,358	49,996,494	4.0257	56.02	4.0370

4 Results

4.1 Controlled 64M Ablation

Table 2 reports the cleanest comparison in this report: four 6-layer runs trained on the same A10 setup with matched effective batch and token budget.

Three conclusions are supported by this controlled comparison.

1. **Sparse routing explains the largest gain.** Standard MoE reduces validation perplexity from 67.68 to 59.33, a 12.3% relative improvement over the dense baseline.
2. **Mediator collaboration plus iterative refinement adds value on top of MoE.** Fixed-depth MoC further reduces perplexity from 59.33 to 55.67, a 6.2% relative improvement over standard MoE at nearly matched active parameters per token.
3. **Adaptive MoC remains close to the fixed-depth variant while learning variable internal compute.** The adaptive run finishes slightly behind the best fixed-depth perplexity (56.02 vs. 55.67, a 0.6% gap), which is acceptable given that its intended purpose is adaptive inference rather than maximum training-set efficiency.

This is a more cautious interpretation than the earlier draft of the paper. On pure validation CE, the fixed-depth MoC run is the strongest 64M model; the adaptive run should be understood as a near-parity variant that exposes a learned compute-allocation policy.

4.2 Adaptive Compute Behavior

The adaptive run does not simply saturate all available internal steps. Instead, it learns a non-uniform pattern:

- average reasoning depth: approximately 2.54 / 3,
- average collaboration rounds: approximately 0.47 / 2,
- estimated internal compute saved: approximately 39.8%.

This asymmetry is informative. At the tested scale, the model appears to prefer spending budget on deeper intra-expert refinement rather than on repeated mediator rounds. That pattern is consistent with a plausible small-scale regime in which experts are only moderately specialized, so additional per-expert computation has higher marginal value than extra cross-expert interaction.

Table 3: Updated-implementation results. These runs are informative, but they are not as clean as the controlled A10 ablation because both code and hardware changed.

Run	Layers	Total Params	Active/Token	Val CE	Val PPL
Current MoC, same scale	6	64,134,540	49,974,156	4.0149	55.42
Current MoC, larger config	8	197,236,240	93,402,640	3.8228	45.73

4.3 Routing Health

Routing diagnostics are healthy across the controlled runs:

- no dead experts were observed in any MoE or MoC layer,
- drop rate remained at zero in the reported 64M runs,
- Gini values stayed low, indicating that utilization was not collapsing onto a single expert,
- the adaptive run preserved similar routing health while adding internal step gates.

These indicators matter because novel sparse architectures often fail for boring reasons before they fail for interesting ones. Here, the system at least clears the boring-failure bar.

4.4 Current Implementation Refresh

Table 3 summarizes the later runs from the updated code path.

The same-scale refreshed run is only modestly better than the earlier fixed-depth 64M MoC result (55.42 vs. 55.67 perplexity). That is still useful: it suggests that the cleaned-up implementation preserved or slightly improved quality rather than destabilizing it. However, because the current code path also changes optimization behavior and data plumbing, this should not be advertised as a pure architecture-only gain.

The larger 197M run is more striking. It reaches validation perplexity 45.73, while maintaining healthy routing statistics across 8 layers: zero dead experts, essentially zero drop, and no obvious utilization collapse. This does *not* prove that MoC beats an equally large MoE baseline - that comparison has not yet been run - but it does provide evidence that the architecture scales beyond the original 64M setting without immediately falling apart.

4.5 What Is Actually Supported Today

Taken together, the evidence supports the following bounded claims:

- Sparse routing is the dominant source of improvement over a dense baseline in this regime.
- Mediator-based collaboration and iterative refinement provide an additional gain on top of standard MoE at 64M scale.
- Adaptive compute is currently supported as a *behavioral* result (the model learns variable internal budget allocation) rather than as a completed efficiency result; end-to-end inference benchmarks are still needed.

- The current MoC implementation scales to a substantially larger sparse model without routing collapse.

5 Related Work

MoC is situated at the intersection of four lines of work.

Sparse MoE scaling. Prior work established sparse expert routing as an effective scaling strategy, including Switch Transformer [1], GShard [2], Mixtral [3], and DeepSeek-MoE [4]. These methods primarily focus on routing and specialization with expert outputs merged after independent processing.

Alternative token–expert interaction schemes. Soft MoE replaces discrete top-k dispatch with a fully differentiable soft assignment in which each expert receives weighted combinations of tokens and returns soft mixtures back to the sequence [11]. That makes Soft MoE especially relevant here: it also changes how experts and tokens interact, but it does so by smoothing dispatch itself. MoC instead keeps sparse top-k routing and adds post-selection collaboration among the chosen experts through a mediator state.

Cross-expert information transfer. HyperMoE shows that expert knowledge can be supplemented using information from unselected experts via hypernetwork-generated transfers, rather than treating experts as completely isolated branches [12]. MoC pursues a different path: rather than injecting information from all experts, it restricts interaction to the selected set and routes messages through a shared mediator hub, preserving linear complexity in top-k experts.

Adaptive compute. Adaptive-depth and adaptive-halting ideas appear in Universal Transformer [5], PonderNet [6], and CALM [7]. MoC adopts this principle inside sparse expert layers and collaboration modules.

Iterative/implicit reasoning. Weight-shared iterative refinement connects to deep equilibrium perspectives [8]. Emerging latent-reasoning directions such as Coconut [9] and pause-token delayed-compute methods [10] further motivate allocating extra internal computation before committing outputs.

To our knowledge, MoC is among the first implementations combining sparse routing, explicit mediator collaboration over selected experts, iterative expert refinement, and adaptive compute gating in one unified decoder architecture.

6 Limitations and Future Work

6.1 Limitations

This report is intentionally conservative about its current evidentiary strength:

- the clean ablation is still limited to a small 64M regime with a single seed,
- the controlled comparison uses one dataset and one training budget only,
- validation is based on 20-batch checkpoints rather than a large held-out sweep,
- the current same-scale refresh mixes architecture, optimizer, data-pipeline, and instrumentation changes,

- the 197M run is a scaling probe, not a controlled comparison against an equally sized MoE baseline,
- training-speed comparisons across A10 and L40S hardware are not meaningful,
- adaptive-compute benefits have been measured internally (steps used, estimated compute saved) but not yet with matched end-to-end inference latency or FLOP benchmarks,
- evaluation is presently limited to language-modeling loss and perplexity; no downstream reasoning or coding benchmark suite is included yet,
- the largest run reported here still uses a relatively short context length and modest expert count compared with modern frontier sparse models.

6.2 Future Work

Priority next steps are straightforward:

- run paired same-size comparisons between current MoC and a standard MoE baseline at larger scale,
- add multi-seed trials and confidence intervals for the 64M ablation,
- benchmark adaptive inference directly with latency, tokens/s, and quality-versus-compute curves,
- test larger expert pools and grouped-GEMM execution paths,
- evaluate on downstream tasks (for example MMLU, GSM8K, HumanEval, and coding benchmarks),
- extend to longer context windows and additional datasets.

7 Conclusion

MoC is best understood, at this stage, as a promising sparse-architecture direction rather than a finished replacement for standard MoE. In a controlled 64M ablation, fixed-depth MoC improves validation perplexity over a matched standard MoE baseline, indicating that mediator-based expert collaboration and iterative refinement are doing something useful beyond ordinary sparse routing. The adaptive variant does not win the 64M quality contest outright, but it does learn a non-trivial compute-allocation policy while staying close to the fixed-depth model, which is exactly the kind of result one would want before spending effort on dedicated inference benchmarking.

The current implementation also looks healthier than the earliest version: same-scale quality is slightly improved, and a larger 197M sparse configuration trains without routing collapse and reaches substantially better validation perplexity. The important caveat is that scaling evidence is not yet the same thing as a controlled architecture-only win. The next decisive experiments are therefore clear: paired larger-scale MoE baselines, direct adaptive-inference benchmarks, and multi-seed repetitions. Until then, the strongest defensible claim is simple: MoC appears real, stable, and worth continuing.

8 Reproducibility Statement

- Code repository: https://github.com/Auren-Research/lunaris
- Experiment logs: https://wandb.ai/smeryylle-moon-cloud-services-/lunaris-moc-validation
- Main branch status: intended to track the current implementation discussed in Section 4.4; earlier controlled ablation runs remain documented in W&B.
- Hardware used in this report: NVIDIA A10 24GB for the controlled 64M ablation and NVIDIA L40S 48GB for the later implementation-refresh and scaling runs.
- Exact configs and code paths: model and training scripts corresponding to the runs discussed in this report are public in the repository and W&B logs.

References

[1] William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120):1-39, 2022.

[2] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Zhou, Zhifeng Chen, Weizhu Cao, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations*, 2021.

[3] Albert Q. Jiang et al. Mixtral of Experts. arXiv preprint arXiv:2401.04088, 2024.

[4] Damai Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv preprint arXiv:2401.06066, 2024.

[5] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations*, 2019.

[6] Andrea Banino, Jan Balaguer, Claudius Voelcker, Jonathan Schwarz, et al. PonderNet: Learning to Ponder. In *International Conference on Machine Learning*, 2021.

[7] Tal Schuster et al. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems*, 2022.

[8] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *Advances in Neural Information Processing Systems*, 2019.

[9] Shibo Hao et al. Coconut: Training Large Language Models to Reason in a Continuous Latent Space. arXiv preprint arXiv:2412.06769, 2024.

[10] Sachin Goyal et al. Think Before You Speak: Training Language Models with Pause Tokens. arXiv preprint arXiv:2310.02226, 2024.

[11] Joan Puigcerver, Carlos Riquelme, Basil Mustafa, and Neil Houlsby. From Sparse to Soft Mixtures of Experts. In *International Conference on Learning Representations*, 2024.

[12] Hao Zhao, Zihan Qiu, Huijia Wu, Zili Wang, Zhaofeng He, and Jie Fu. HyperMoE: Towards Better Mixture of Experts via Transferring Among Experts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10556–10571, 2024.